



Deploying UnityCloud on OCP

Release UC5.2.0

Date: May 2025

Affirmed Networks, Inc.

A Microsoft Company

Deploying UnityCloud on OCP
Revision: A

Notices

Copyright© 2025 Affirmed Networks, Inc. All Rights Reserved.

AFFIRMED & DESIGN®, ACUITAS SERVICE MANAGEMENT SYSTEM®, AFFIRMED NETWORKS MOBILE CONTENT CLOUD™, AFFIRMED MOBILE CONTENT CLOUD™, UnityCloud™, UnityCloud PaaS™, UnityCloud Platform™, UnityCloud Operations™, UnityCloud Ops™, Operations and Policy Manager (OPM)™ and other trademarks and designs are the registered or unregistered trademarks of Affirmed Networks, Inc. and its subsidiaries in the United States and in foreign countries. All other trademarks are the property of their respective owners. The Affirmed Networks, Inc. trademarks may not be used in connection with any product or service that is not Affirmed Networks' in any manner that is likely to cause confusion among customers or in any manner that disparages or discredits Affirmed Networks, Inc.

This document contains information that is the property of Affirmed Networks, Inc. This document may not be copied, reproduced, reduced to any electronic medium or machine readable form, or otherwise duplicated, and the information herein may not be used, disseminated or otherwise disclosed, except with the prior written consent of Affirmed Networks, Inc.

Affirmed NETWORKS, INC. SOFTWARE LICENSE AGREEMENT

Affirmed NETWORKS, INC. ("Affirmed") IS WILLING TO LICENSE THE ENCLOSED SOFTWARE AND ACCOMPANYING USER DOCUMENTATION (COLLECTIVELY, THE "PROGRAM") TO YOU ONLY UPON THE CONDITION THAT YOU ACCEPT ALL OF THE TERMS AND CONDITIONS OF THIS LICENSE AGREEMENT. PLEASE READ THE TERMS AND CONDITIONS OF THIS LICENSE AGREEMENT CAREFULLY BEFORE OPENING THE PACKAGE (S) OR USING THE Affirmed PRODUCTS CONTAINING THE SOFTWARE, AND BEFORE USING THE ACCOMPANYING USER DOCUMENTATION. OPENING THE PACKAGE (S) OR USING THE Affirmed PRODUCTS CONTAINING THE PROGRAM WILL INDICATE YOUR ACCEPTANCE OF THE TERMS OF THIS LICENSE AGREEMENT. IF YOU ARE NOT WILLING TO BE BOUND BY THE TERMS OF THIS LICENSE AGREEMENT, Affirmed IS UNWILLING TO LICENSE THE PROGRAM TO YOU, IN WHICH EVENT YOU SHOULD RETURN THE PROGRAM WITHIN TEN (10) DAYS FROM SHIPMENT TO THE PLACE FROM WHICH IT WAS ACQUIRED, AND YOUR LICENSE FEE WILL BE REFUNDED. THIS LICENSE AGREEMENT REPRESENTS THE ENTIRE AGREEMENT CONCERNING THE PROGRAM BETWEEN YOU AND Affirmed, AND IT SUPERSEDES ANY PRIOR PROPOSAL, REPRESENTATION OR UNDERSTANDING BETWEEN THE PARTIES.

License Grant

Subject to the provisions of this License, Affirmed grants to Licensee, and Licensee accepts, a non-exclusive, non-transferable license to use the object code form of the software supplied by Affirmed, including all patches, error corrections, updates and revisions thereto in machine-readable, object code form only (the "Software"), and the user documentation, including all updates and revisions thereto (the "User Documentation") for Licensee's internal business purposes (including, without limitation, in conjunction with Licensee's provision of services to its customers) on or in conjunction with the hardware product with which it was originally delivered or on a single designated computer. The Software and User Documentation are sometimes collectively referred to as the "Program". *Licensee acknowledges that certain Software provided by Affirmed may contain other restrictions and Licensee must refer to accompanying license certificates for each such Software licensed.* Licensee agrees that Licensee will not pledge, lease, rent, or share Licensee's rights under this License Agreement, and that Licensee will not, without Affirmed's prior written consent, assign or transfer Licensee's rights hereunder. Licensee agrees that Licensee may not duplicate (except as set forth below), decompile, disassemble, reverse engineer, modify, or otherwise translate the Software or permit a third party to do so and that Licensee shall not make the Program available to any other third party, or create works derivative of the Program, without Affirmed's express written consent. Licensee agrees it shall not publish in any fashion any results of benchmark tests run on the Program.

The Program is copyrighted and is only authorized to reproduce one copy of the Software and the User Documentation solely for backup purposes. Licensee is hereby prohibited from otherwise copying or translating, modifying or adapting the Program or, incorporating in whole or any part in any other product or creating derivative works based on all or any part of the Program. Licensee is not authorized to license others to reproduce any copies of the Program, except as expressly provided in this Agreement. Licensee agrees to ensure that all copyright, trademark and other proprietary notices of Affirmed affixed to or displayed on the Program will not be removed or modified. Licensee shall not decompile, disassemble or reverse engineer, the licensed software or any component thereof, except as may be permitted by applicable law, in which case Licensee must notify Affirmed in writing and Affirmed may provide review and assistance.

U.S. Government Restricted Rights. Notice - Distribution and use of products including computer programs and any related documentation and derivative works thereof, to and by the U.S. Government, are subject to the Restricted Rights provisions of FAR 52.227-19, paragraph (c)(2) as applicable, except for purchases by agencies of the Department of Defense (DOD). If the software is acquired under the terms of a Department of Defense or civilian agency contract, the software is a "commercial item" as that term is defined at 48 C.F.R. 2.101 (Oct. 1995), consisting of "commercial computer software" and "commercial computer software documentation" as such terms are used in 48 C.F.R. 12.212 of the Federal Acquisition Regulations and its successors and 48 C.F.R. 227.7202-1

through 227.7202-4 (June 1995) of the DoD FAR Supplement and its successors. All U.S. Government end users acquire the software with only those rights set forth in this Agreement. Manufacturer is Affirmed Networks, Inc., . . . Unpublished - rights reserved under the copyright laws of the United States.

Affirmed's Rights

Licensee agrees that the Software and the User Documentation are proprietary, confidential products of Affirmed or Affirmed's licensor protected under U.S. copyright law and Licensee will use Licensee's best efforts to maintain their confidentiality. Licensee further acknowledges and agrees that all right, title and interest in and to the Program, including associated intellectual property rights, are and shall remain with Affirmed or Affirmed's licensor. This License Agreement does not convey to Licensee an interest in or to the Program, but only a limited right of use revocable in accordance with the terms of this License Agreement.

License Fees

The license fees paid by Licensee are paid in consideration of the license granted under this License Agreement.

Term

This License Agreement is effective upon Licensee's execution of this License Agreement and payment of the license fees and shall continue until terminated. Licensee may terminate this License Agreement at any time by returning the Program and all copies or portions thereof to Affirmed. Affirmed may terminate this License Agreement upon the breach by Licensee of any term hereof. Upon such termination by Affirmed, Licensee agrees to return to Affirmed the Program and all copies or portions thereof. Termination of this License Agreement shall not prejudice Affirmed's rights to damages or any other available remedy.

Limited Warranty

Affirmed warrants that with normal use and service each Software product shall materially conform to Affirmed's then current published specifications for the most current release of such Software product for a period of ninety (90) days from the date of shipment by Affirmed. Program media is warranted for ninety- (90) days from the date of shipment by Affirmed. Software support beyond these periods is available at additional cost under the terms of Affirmed's Annual Maintenance Service Agreement. During the warranty period, as Licensee's sole and exclusive remedy, Affirmed will correct a Software product's failure to conform to the warranty provided that Licensee has notified Affirmed in writing of the nature of the non-conformity. This warranty shall not apply if any Software product has been (i) modified or altered by anyone other than Affirmed, (ii) abused or misapplied, or (iii) used in combination with hardware or software other than the Affirmed manufactured products for which it was designed. Affirmed shall incur no liability under this warranty if Affirmed's tests disclose that the alleged defect is due to causes not within Affirmed's reasonable control, including alteration or abuse of the goods. If a Program is determined not to be defective or to have a defect due to causes not within Affirmed's reasonable control, Licensee agrees to pay for such repair at the repair price as listed in Affirmed's then current applicable price list. In no event does Affirmed warrant that the use of Software products will be error free or uninterrupted. Affirmed's sole obligation under the Software warranty shall be to provide the remedies described above.

EXCEPT FOR THE EXPRESS WARRANTIES STATED IN THIS WARRANTY, THE PROGRAM IS LICENSED "AS IS", AND Affirmed DISCLAIMS ANY AND ALL OTHER WARRANTIES, WHETHER EXPRESS, IMPLIED OR STATUTORY WITH RESPECT TO THE SOFTWARE PROVIDED UNDER THIS AGREEMENT, INCLUDING, BUT NOT LIMITED TO, ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE, ANY WARRANTIES OF NONINFRINGEMENT AS WELL AS ANY WARRANTIES ARISING FROM COURSE OF DEALING, USAGE OR TRADE PRACTICE.

Limitation of Liability

Affirmed's cumulative liability to Licensee or any other party for any loss or damages resulting from any claims, demands, or actions arising out of or relating to this License Agreement shall not exceed the greater of: (i) ten thousand U.S. dollars (\$10,000) or (ii) the total license fee paid to Affirmed for the use of the Program. In no event

shall Affirmed be liable for any indirect, incidental, consequential, special, punitive or exemplary damages or lost profits or the loss of software or data, even if Affirmed has been advised of the possibility of such damages. In the event that the Program contains the intellectual property of a third party pursuant to a license in favor of Affirmed, with a right to sublicense to Licensee, Affirmed hereby disclaims, to the extent permitted by law, such third party's liability of any damages, whether direct, indirect, incidental or consequential, arising out the use of the Program

Export Control

Licensee shall not export or re-export, directly or indirectly, any of the Programs (including any part of a Program or any direct product of such Program), Affirmed's Confidential Information or related technical data (a) into, or to a national resident of, any of those countries listed at the time of any shipment of the Products in the applicable U.S. export regulations as "prohibited or restricted" countries, or any other country to which such exports or re-exports may be restricted (collectively, the "Prohibited Countries") or (b) to anyone on the U.S. Treasury Department's list of Specially Designated Nations or the U.S. Commerce Department's Table of Denial Orders. At Licensee's request, Affirmed shall provide a list of the Prohibited Countries, Specially Designated Nations and/or Table of Denial Orders in order to assist Licensee to comply with such requirement.

Governing Law

This License Agreement shall be construed and governed in accordance with the laws and under the jurisdiction of the Commonwealth of Massachusetts, U.S.A. Any dispute arising out of this Agreement shall be referred to an arbitration proceeding in Boston, Massachusetts, U.S.A. by the American Arbitration Association.

Miscellaneous

If any action is brought by either party to this License Agreement against the other party regarding the subject matter hereof, the prevailing party shall be entitled to recover, in addition to any other relief granted, reasonable attorneys' fees and expenses of arbitration. Should any term of this License Agreement be declared void or unenforceable by any court of competent jurisdiction, such declaration shall have no effect on the remaining terms hereof. The failure of either party to enforce any rights granted hereunder or to take action against the other party in the event of any breach hereunder shall not be deemed a waiver by that party as to subsequent enforcement of rights or subsequent actions in the event of future breaches.

Affirmed has made no commitments or promises orally or in writing with respect to delivery of any future software features or functions. In relation to any future software features or functions, all presentations, RFP responses and/or product outlook documents, information or discussions, either prior to or following the date herein, are for informational purposes only, and Affirmed has no obligation to provide any future releases or upgrades or any features, enhancements or functions, unless specifically agreed to in writing by both parties. Reseller acknowledges that no purchasing decisions are based upon any future software features or functions.

Third Party Beneficiary

If the Program incorporates or otherwise contains any intellectual Property of any third party pursuant to a license agreement in favor of Affirmed and sublicensed to Licensee, such third party shall, to the extent permitted by law, be a third party beneficiary of the terms and conditions of this Agreement.

Contents

Notices	2
Preface	11
Audience	11
Release Information	11
Conventions	11
How to Obtain Product Documentation	12
Customer Support	12
Prerequisites	13
Post-Installation Configuration	15
Worker Nodes	15
Dataplane Nodes	16
Cluster Configuration	16
Dataplane Node Configuration	16
Node Feature Discovery and SR-IOV Operator	18
Deploying OPM	21
Prerequisites	21
OPM Deployment Procedure	21
Deploying Federations	24
Download charts	24
Federations replaced by OCP features	24
Deploying Federations With OPM	25
Chart Installation Order	25
Installing a Federation	25
Optional Observability Features	26
Deploying Federations Without OPM	27
Log into OCP	27
(Optional) Provide image pull permission	27
Configure and install common federations	28
Install or Enable Observability Components	31
Install Network Function Federations	33
Configuring deployed network functions	34
Helm Values For UnityCloud Federations	35
Common Federations	35
fed-crds	35
fed-metallb	35
fed-opa	35
fed-istio	35
fed-paas_helpers	36

fed-kubedb_operator	37
fed-service-reg	37
fed-db_etcd	37
fed-db_mongo_ram	38
Observability Federations	38
fed-prometheus	38
fed-grafana	38
Network Federations	38
fed-smf	38
fed-amf	38
fed-nrf	39
fed-nssf	39
fed-upf	39

List of Tables

Table 7-1 Required Helm Charts	24
--------------------------------------	----

Preface

This guide describes how to deploy UnityCloud on OpenShift Container Platform (OCP).

Audience

This guide is intended for network operators (sometimes referred to as Users or Operators in this guide), installation team staff, and other qualified service personnel responsible for installing, provisioning, and monitoring the system.

The procedures in this guide require you to understand and follow the safety practices at your site, as well as those identified in this guide.

Release Information

See the UnityCloud™ Software Release Notes for important information, requirements, limitations, and restrictions that apply to this release.

Conventions

This guide uses the following conventions, when applicable:

Description	Convention	Examples
Command names, keywords, dialog boxes, buttons, icons, menu	Times New Roman bold font	Use configure_ha to configure... Enter y . To continue, press Enter .
Selecting a menu item	Menu => Option	Select Security => Group Management
Variable parameters for which you supply values	<courier bold italic>	configure_dr --host <host name>
User input	Courier bold font	cd /opt/Affirmed/NMS/bin
Screen messages, system output, sample source code	Courier regular font	result success
Required keywords and arguments in square brackets	[]	cluster [id] node [id] reboot
Optional keywords and arguments in curly brackets	{ }	{cdr-file-name}
Keywords separated by the pipe symbol. Requires you to specify only one item from the set.		Is this correct? [y n]

Description	Convention	Examples
Keywords representing a single element in angle brackets	< >	<host name>
Asterisk after a CLI object indicates a required object	*	name*
Variables, book and chapter titles, file path names, new terms, and emphasized text	<i>Times New Roman italic font</i>	See the <i>Deploying UnityCloud on OCP</i> . <i>Workflow</i> is an application running on the network element.
Images in online help		Many images can be enlarged by clicking the image with the left-mouse button. Alternately, they can be enlarged/reduced by pressing the Enter key when the image is highlighted (surrounded by a thick border) on the screen.

Note: Additional information that may apply to the subject text.

Tip: Helpful suggestions.

Caution: Proceed carefully to avoid possible equipment damage or data loss.

How to Obtain Product Documentation

Documentation is available for currently supported product releases. You can access Affirmed Networks Guides and Release Notes on the [Manuals](#) site. From here, you can also download a PDF version of an individual guide, or the full manual set for a product. For access and instructions, please contact Support.

Customer Support

Affirmed Networks provides technical support 24 hours a day, 7 days a week (24 x 7) for any customer with an active maintenance contract.

[Email support](#)

[Open a support ticket](#)

Call support

Local: +1-978-268-0800

Toll-free: +1-888-283-2838

Prerequisites

Your environment must meet the following requirements in order to deploy UnityCloud on OpenShift Container Platform (OCP).

OCP 4.6

OVN-Kubernetes is used as the CNI plugin. Red Hat Enterprise Linux CoreOS (RHCOS) is used for the compute nodes.

To deploy an OCP cluster on bare metal, please refer to Red Hat's official OpenShift documentation here: [Installing a cluster on bare metal with network customizations](#).

External NFS Server

An external NFS server is required in order to persist configuration manager XML files, capture files, and core dumps.

If using OpenShift Container Storage or similar, the file system storage class can be used to dynamically create persistent volumes (PVs) for persisting Network Function configuration files. NFS is required for packet capture files and core dumps.

***Note:** If using OPM that is deployed as a VM, its built-in NFS server may be used.*

(Optional) Openshift Container Storage (OCS) with storage class for file and block storage

OCS CephFS storage can be used for Network Function configuration data.

(Optional) External Chart Repository

You may deploy charts from a repository if not installing the chart packages directly.

BGP-enabled router load balancing across UnityCloud nodes

This includes a reserved IP range and ASN to configure in MetalLB for advertising external service IP addresses.

***Note:** OpenShift Routes are not used with UnityCloud Network Functions.*

UnityCloud container images in an accessible registry

UnityCloud container images must already have been pushed to the site's image registry.

Prerequisites for UPF deployment

A UPF deployment requires:

- Cluster with SR-IOV interfaces and minimum of 16 CPUs/node and 32 GB of memory
- A minimum of two worker nodes
- Support for hugepages
- UPF federation installation.

Post-Installation Configuration

This section describes the post-installation configuration required to run UnityCloud Network Functions on OCP. OCP post-install commissioning is achieved by applying MachineConfigs (MC), Tuned profiles and Kubelet configs on the cluster. These 3 artifacts are applied to a MachineConfigPool (MCP). An MCP is a subset of OCP machines (nodes) requiring additional post-install configuration. For example, a subset of servers can be identified as dataplane-capable nodes. By default, UnityCloud worker nodes run control-plane workloads. This means that a MCP can be assigned to a node that runs both control-plane and dataplane workloads. MCs are preferably applied before deploying CNFs because they require draining and rebooting of nodes.

For dataplane, the node(s) are assigned the 'dataplane' role associating them to the dataplane MCP, which updates these nodes with kernel args, hugepages, kubelet settings, etc.

Worker Nodes

The post-cluster configurations in this section apply to both control plane and dataplane nodes:

1. Enable the following sysctl: `net.ipv4.conf.all.route_localnet`

Note: Please refer to Red Hat's official documentation on enabling unsafe sysctls: [Enabling unsafe sysctls](#).

The following example Machine Config may be applied to worker nodes:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: custom-kubelet
spec:
  machineConfigPoolSelector:
    matchLabels:
      route_localnet: "true"
  kubeletConfig:
    allowedUnsafeSysctls:
      - "net.ipv4.conf.all.route_localnet"
```

2. Increase CRI-O and Kubelet default Pod PIDs to 10240.

The following example Machine Configs may be applied to worker nodes:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: KubeletConfig
metadata:
  name: pod-pids-limit
spec:
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet-pod-pids-limit: 'true'
  kubeletConfig:
    PodPidsLimit: 10240
---
```

```
apiVersion: machineconfiguration.openshift.io/v1
kind: ContainerRuntimeConfig
metadata:
  name: set-pids-limit
spec:
  machineConfigPoolSelector:
    matchLabels:
      custom-kubelet-pod-pids-limit: 'true'
  containerRuntimeConfig:
    pidsLimit: 10240
```

3. SCTP should be left disabled in the cluster if you plan to deploy fed-amf.

Dataplane Nodes

Cluster Configuration

Perform the following steps:

1. Enable the latency sensitive configuration for performance (needed for topology manager):

```
# oc patch featuregate cluster --type=merge --patch '{"spec": {
"featureSet": "LatencySensitive" } }'
```
2. Verify configuration. Wait for the state below before moving on to the next step. This may take 15 minutes or more:

```
# watch oc get mcp
```

Expected for dataplane-mcp:

```
UPGRADED = True, UPDATING & DEGRADED = False,
DEGRADEDMACHINECOUNT = 0
```
3. Run the following command:

```
# oc get nodes
```

Expected: All nodes Ready, no SchedulingDisabled.

Dataplane Node Configuration

Perform the following steps:

1. Identify all the workers capable of running dataplane/workers you want to use for dataplane and for each, run the following command:

```
# oc label node <worker-name> noderole.kubernetes.io/dataplane
```

Verify with the following command:

```
# oc get nodes
```

Expected: `worker` in the role of the targeted worker node, along with `dataplane`.

Note: A role can be removed from workers later if needed using:

```
# oc label node <worker-name> node-role.kubernetes.io/<role-name>-
```

- Using the provided sample config files, create new MachineConfig, Tuned profile and KubeletConfig resources to setup additional dataplane/sriov-specific node commissioning parameters (kernel args, kernel modules, hugepages, kubelet config, cpu manger, topology manager, etc.):

```
# oc create -f ocp-dataplane-machine-config.yaml ; oc create -f ocp-dataplane-tuned-profile.yaml ; oc create -f ocpdataplane-kubelet-config.yaml
```

Note: Review the above yaml files and confirm that all arguments/values are applicable to your setup/environment (or modify as needed). The above changes will be applied to the dataplane nodes ONLY after the MachineConfigPool section below is completed. The kubelet config's topology manager is currently set to 'best-effort'.

- Using the provided config files, create a new MCP that will apply the above MachineConfig and Tuned profiles to the dataplane nodes:

```
# oc create -f ocp-dataplane-machine-config-pool.yaml
```

Note: Review the above yaml files and confirm that all arguments/values are applicable to your setup/environment (or modify as needed). This MCP yaml is designed to be applied on nodes that have the 'dataplane' role assigned above. The MCP can be paused/unpaused with:

```
# oc patch mcp dataplane-mcp --type=merge --patch '{ "spec": { "paused": true } }', oc patch mcp dataplane-mcp -- type=merge --patch '{ "spec": { "paused": false } }'
```

- Verify configuration. To monitor MCP progress while it is applied:

```
# watch oc get mcp
```

Expected for dataplane-mcp:

```
UPGRADED = True, UPDATING & DEGRADED = False, MACHINECOUNT,
READYMACHINECOUNT & UPDATEDMACHINECOUNT = nb-dataplanenodes,
DEGRAEDMACHINECOUNT = 0
```

```
dataplane-mcp rendered-dataplane-
mcp24c3afec54c60f28610700af951cbee4 True False False 1 1 1 0
55m
```

If any issue is observed, the following can be useful to understand what is blocking machineConfigs progress:

```
# oc describe mcp dataplane | grep Message
# oc logs -f <machine-config-controller-name> -n
openshiftmachine-config-operator

# oc logs -f <machine-config-daemon-name> -n openshiftmachine-
config-operator machine-config-daemon
```

Where **<machine-config-daemon-name>** is the name of the daemon running on a targeted dataplane node.

5. Check that the MachineConfigs includes the Tuned profile and KubeletConfig:

```
# oc get machineconfigs | grep dataplane
```

6. Verify that the MCP has selected all 3 configurations created above:

```
# oc describe mcp dataplane-mcp | grep Name:
```

7. Confirm that 'dataplane' nodes are now excluded from the count of 'worker' MCP:

```
# oc describe node <dataplane-node-name> | grep rendered
```

Expected: currentConfig = desiredConfig

8. Verify that the expected configurations have been applied to the nodes. Log into each dataplane node using:

```
# oc debug node/<worker-name>
```

and then **# chroot /host** to get access to restricted commands.

9. Verify that the KubeletConfig has been applied:

```
# cat /etc/kubernetes/kubelet.conf | grep -e cpuManager -e
topologyManager
```

```
"cpuManagerPolicy": "static",
"cpuManagerReconcilePeriod": "10s",
"topologyManagerPolicy": "single-numa-node",
```

```
# cat /proc/cmdline
```

Confirm the following is present:

```
intel_iommu=on iommu=pt intel_iommu=on iommu=pt hugepagesz=2M
hugepages=8192
```

```
# lsmod | grep vfio
```

```
vfio_iommu_type1 32768 0
vfio 36864 1 vfio_iommu_type1
```

Node Feature Discovery and SR-IOV Operator

The Single Root I/O Virtualization (SR-IOV) Network Operator manages the SR-IOV network devices and attachments in the cluster. This needs to be configured to support UnityCloud dataplane nodes.

1. Verify if the Node Features Discover operator is already deployed:

```
# oc get csv -n openshift-operators -o custom-
columns=Name:.metadata.name,Phase:.status.phase | grep nfd
```

Expected: nfd.4.6.0-202101160045.p0 Succeeded

***Note:** If the nfd operator is not present, use the [web console installation](#) but search and install nfd operator from the marketplace.*

2. Verify if the SRIOV network operator is already deployed:

```
# oc get csv -n openshift-sriov-network-operator -o custom-
columns=Name:.metadata.name,Phase:.status.phase
```

Expected:

Name Phase

sriov-network-operator.4.6.0-202012160125.p0 Succeeded

***Note:** If the above sriov-network-operator is not present, use the following to install it: [web console installation](#) (not CLI)*

3. Use the following command to observe the different SRIOV Network Operator CRDs:

```
# oc get crd -A | grep sriov
```

4. The following command shows all the worker's NICs along with their characteristics. Identify the ones that will be used for dataplane among the list obtained by running the following:

```
# oc describe sriovnetworknodestate <worker-name> -n openshift-
sriov-network-operator
```

5. Using the provided file, apply the policy needed to prepare/configure the above SRIOV ports for usage in dataplane pods. This publishes the SRIOV VFs as Kubernetes allocatable node resources.

- a. For each dataplane node, log into the node using:

```
# oc debug node/<worker-name> and then # chroot /host to get access to
restricted commands and cd ~
```

- b. Apply config to each dataplane node

```
# scp bld-dataplane_base/working/cni-
infradataplane/prototype/baremetal/openshift/ocp-
machineconfigs/tmp-vfio-setup.sh
```

to the node to apply the workaround configuration to each dataplane nodes.

- c. Create bind.sh based on the instructions in comment in tmp-vfio-setup.sh and then execute tmp-vfio-setup.sh. This needs to be applied after every node reboot.

Using the yaml below, manually create the sriovnetworknodepolicy using:

Note: Adapt based on the SRIOV dataplane ports in your setup. The yaml below, for example, created a 'sr1' pool of VFs from ens4f0 Fortville 710 158b port and 'sr2' from ens4f1

```
# oc create -f fvl-openshift-sriov-node-network.yaml
```

Verification:

```
# oc get sriovnetworknodepolicies.sriovnetwork.openshift.io -A
```

should show 'sr1' and 'sr2' (assuming the Intel 710 158b from setup)

```
# oc describe node <worker-name>
```

and look at the Allocatable section, openshift.io/sr1 and openshift.io/sr2: should be present with number of VFs available in each pool respectively. It may take several minutes before these values are observable.

Deploying OPM

An OPM instance can deploy federations on OCP by configuring the OCP cluster as an "Existing" cluster and providing the kubeconfig file generated during OCP cluster deployment. The OPM instance can either be external to the OCP cluster (for example, running as a VM) or be made to run as a workload on the OCP cluster. The following steps enumerate how to deploy OPM as a workload on OCP so it can be used to configure and deploy federations as instructed in the UnityCloud Deployment Guide.

Prerequisites

OPM requires access to the host filesystem (hostPath mounts) or NFS mounts to persist its data. By default, OCP clusters use CoreOS hosts, which have an ephemeral filesystem. This means OPM cannot use hostPath mounts without configuring additional disks on the host to be mounted at boot time. To be able to deploy OPM on OCP, an NFS server is required.

In order to deploy OPM, the following federations must first be deployed. For instructions on how to deploy federations without OPM, see [Deploying Federations Without OPM](#)

- fed-crds
- fed-rbac

Additionally, the following directories need to be pre-created on the NFS server's root share and given 777 permissions:

***Note:** This step is temporary and will be removed in a future update.*

- /vnfmgr/cfg
- /vnfmgr/web
- /vnfmgr/certs
- /vnfmgr/db
- /vnfmgr/dreg
- /vnfmgr/feds
- /chartm-storage

OPM Deployment Procedure

1. Create vnfmgr project and provide necessary permissions.

```
# oc new-project vnfmgr
```
2. Provide following escalations to the vnfmgr project

```
# oc adm policy add-scc-to-group privileged
system:serviceaccounts:vnfmgr

# oc adm policy add-scc-to-group hostmount-anyuid
system:serviceaccounts:vnfmgr

# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:vnfmgr
```

3. Prepare Helm values.yaml file to be used in deploying fed-opm via Helm: Create fed-opm-values.yaml file. See sample file below. Replace `<external ip>`, `<nfs server>` and `<registry>` accordingly. Replace `mongoServerImage` with most recent image.

```
global:
  defaultClusterRole: permissive-full-cr
  defaultRbacEnabled: true
  defaultServiceAccountName: unity-cloud-serviceaccount
  externalIPs: "<external ip>"
  registry:
    docker:
      repoPath: "<registry>"
  fileserver:
    nfs:
      remoteHost: "<nfs server>"
      remotePort: "85"
  service:
    externalIPs: "<external ip>"
  pod-opm_ambassador:
    serviceAccount:
      name: unity-cloud-serviceaccount
  pod-chart_museum_ui:
    chartMuseumIp: "<external ip>"
  pod-opm_master_rm:
    deployment:
      mongoServerImage: centos/mongodb-36-centos7:1
```

4. Deploy fed-opm:

Use Helm3 client. Replace `<fed-opm-chart>` with fed-opm chart URL or package. Use the values file created in last step. To verify template before installing do a `--dry-run` first.

```
# helm install fed-opm --namespace vnfmgr <fed-opm-chart> -f
fed-opm-values.yaml --debug
```

5. Verify deployment. Make sure all pods are running:

```
# oc -n vnfmgr get pods
```

6. OPM should be accessible via the EXTERNAL-IP of the vnfmgr-ambassador service. To check that services are exposed successfully run the following:

```
# oc -n vnfmgr get svc
```

7. Configure OPM

- Optionally, create new user(s)
- Add a location for the OCP cluster
- Add ChartMuseum repository. The service name is vnfmgr-chartm.

- Add an "Existing" deployment zone using the newly added location and chart repository.
 - Add the OCP cluster as an "Existing" Kubernetes cluster within the newly added deployment zone. Provide the kubeconfig file generated during OCP cluster deployment.
8. The OPM can now be used to deploy federations instead of using the Helm client.

Deploying Federations

Download charts

If not using an external Helm repository (e.g. Chart Museum), download all the fed charts into a server that has oc and Helm 3 installed. The server should have access to the OCP machine network and the installed clients configured to access the cluster. If using OPM to deploy the federations, the charts can optionally be pushed to the OPM chart repository.

Table 7-1 Required Helm Charts

Common Federations	Observability Federations	Network Federations
▪ fed-rbac ▪ fed-crds ▪ fed-metallb ▪ fed-opa ▪ fed-istio ▪ fed-paas_helpers ▪ fed-kubedb_operator ▪ fed-service_reg ▪ fed-db_etcd ▪ fed-db_mongo_ram	▪ fed-grafana	▪ fed-smf ▪ fed-upf ▪ fed-amf ▪ fed-nrf ▪ fed-nssf

Federations replaced by OCP features

The following UnityCloud federations are replaced by OCP features when deploying UnityCloud on OCP:

- fed-prometheus: Use [OCP user workload monitoring](#).
- fed-elastic, fed-kibana: Use [OCP cluster logging](#).
- fed-jaeger: Use [OCP jaeger operator](#).

Deploying Federations With OPM

OPM provides a graphical user interface and some automation to aid in the deployment and maintenance of UnityCloud federations. If you have not yet deployed and configured OPM, please refer to [Deploying OPM on OCP](#).

Chart Installation Order

Use OPM to install UnityCloud federations in the following order:

Common federations:

- fed-rbac
- fed-crds
- fed-metallb
- fed-opa
- fed-istio
- fed-paas_helpers
- fed-kubedb_operator
- fed-service_reg
- fed-db_etcd
- fed-db_mongo_ram

Observability Federations:

- fed-grafana

Network Federations:

- fed-smf
- fed-upf
- fed-amf
- fed-nrf
- fed-nssf

Installing a Federation

Perform the following steps to install a federation.

1. From the main OPM screen, click **Install Federation**.

Note: Federations can also be managed from the Deployment Zone View. This section assumes a deployment zone and cluster have already been created.

2. From the **Install Federation** screen, select the Helm repository from the **Select New Chart** drop-down to display a list of charts. To filter the list, enter a filter condition (for example, version number) in the **Keyword Filter** field and click the **Filter results** button.
3. Select a chart from the list and configure the parameters. By default, the system displays the Required parameters from the values.yaml file, which contains the default configuration values for this chart. To display all the configurable values, select **None** from the **Filter** dropdown.

Note: Parameters specific to OCP deployment can be found in the section [Helm Values for UnityCloud Federations](#) of this document.

4. Click **Select Cluster** to display a dialog box from which you can select the cluster to which the federation will be deployed. After you have selected the cluster, the cluster is set for subsequent federation deployments.
5. From the **Select Cluster** dialog box, select the cluster and click **Select**.
6. (Optional) Click the **Dry Run** button (clipboard icon) to verify that the chart will deploy correctly.
7. Click the **Install Federation** button (gear icon) to deploy the chart.

A pop-up window appears after the federation is installed successfully and the microservice status appears. Use the Ellipsis icon (...) to update the federation status. If a federation does not install successfully, you can roll back to a prior version from the **Install Federation** screen.

Optional Observability Features

For information on how to install the following optional observability features that are unavailable for installation through OPM, please refer to [Deploying Federations Without OPM](#):

- OCP Cluster Logging
- OCP Jaeger
- OCP Monitoring for user workload
- OCP Alerting

Deploying Federations Without OPM

The following steps use the Helm3 client from a machine that has access to the OCP cluster as well as the UnityCloud Helm Charts.

Log into OCP

Log in using the token which can be retrieved from the OCP web console:

```
# oc login --token=<token> --server=<ocp_api_server>
```

(Optional) Provide image pull permission

If using OCP's internal registry and images are stored in a project, make sure the images are pullable from projects that use them. Note that to be able to perform the steps below, the projects need to be pre-created. Alternatively, pull permissions can also be provided after deploying a federation.

Example

If images are all stored in `image-registry.openshift-imageregistry.svc:5000/fed-example`, provide `image-puller` permission to pods running in other projects:

```
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-rbac --
namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-metallb --
namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-opa --
namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-istio --
namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-paashelpers
--namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-kubed-
boperator --namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-servicereg
--namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-db-etcd --
namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-db-mongoram
--namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-grafana --
namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-elastalert
--namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-nrf --
namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-nssf --
namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-upf --
namespace=fed-example
# oc policy add-role-to-group system:image-puller system:serviceaccount:fed-amf --
namespace=fed-example
```

Configure and install common federations

After installing a federation, verify that resources are deployed and pods are running before continuing with the next federation. When installing, it is recommended to use a separate Helm values file to override chart values. Please refer to the provided Values files and remember to replace environment specific configuration parameters.

Use **--debug** and **--dry-run** to verify templates and assist in debugging issues.

Use **--create-namespace** to automatically create the namespace and OCP project resource.

***Note:** Some of the procedures in this section require granting privileged permissions to pods running in the associated federation. This is a temporary solution and will be made unnecessary in a future update.*

Install fed-rbac

Configure values in fed-rbac-values.yaml, then install the chart and verify deployment afterward:

```
# helm install fed-rbac --create-namespace --namespace fed-rbac
<chart> --set global.registry.docker.repoPath=<registry_url>
```

Install fed-crds

Edit values in fed-crds-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install the chart and verify deployment afterward:

```
# helm install fed-crds --create-namespace --namespace fedcrds
<chart> -f fed-crds-values.yaml
```

Install fed-metallb

Provide privileged permissions to pods running in fed-metallb:

```
# oc adm policy add-scc-to-group privileged
system:serviceaccounts:fed-metallb
```

Edit values in fed-mb-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterward:

```
# helm install fed-metallb --create-namespace --namespace fed-
metallb <chart> -f fed-mb-values.yaml
```

Install fed-opa

Provide anyuid and privileged permissions to pods running in fed-opa. Make sure to update other required parameters including `global.grafana.url`.

```
# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-opa

# oc adm policy add-scc-to-group privileged
system:serviceaccounts:fed-opa
```

Edit values in fed-opa-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterward:

```
# helm install fed-opa --create-namespace --namespace fedopa
<chart> -f fed-opa-values.yaml
```

Install fed-istio

Provide anyuid permissions to pods running in fed-istio project. This is needed because fed-istio runs containers with user id 1337.

```
# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-istio
```

Edit values in fed-istio-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterwards:

```
# helm install fed-istio --create-namespace --namespace fed-istio
<chart> -f fed-istio-values.yaml
```

Install fed-paas_helpers

Provide privileged, anyuid and host-anyuid permissions to pods running in fed-paashelpers project:

```
# oc adm policy add-scc-to-group privileged
system:serviceaccounts:fed-paas-helpers

# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-paas-helpers

# oc adm policy add-scc-to-group hostmount-anyuid
system:serviceaccounts:fed-paas-helpers
```

Provide fed-paas_helpers default user with admin role so that it can pick up authorization credentials from OCP, so that fed-grafana can connect to OCP Prometheus:

```
# oc adm policy add-cluster-role-to-user admin
system:serviceaccount:fed-paas-helpers:default
```

Edit values in fed-paas-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

***Note:** Make sure variable `podgrafana_operator.deployment.authToPrometheusAuth.enabled` is set to `true`. This allows `fed-grafana` to authenticate with OCP Prometheus.*

Install chart. Verify deployment afterwards:

```
# helm install fed-paas-helpers --create-namespace -- namespace
fed-paas-helpers <chart> -f fed-paasvalues.yaml
```

Install fed-kubedb_operator

Provide anyuid permissions to pods running in `fed-kubedb-operator` project.

```
# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-kubedb-operator
```

Edit values in `fed-kubedb-values.yaml` as shown in [Helm Values for UnityCloud Federations](#). Remember that the mongo image is nonredistributable and needs to be manually configured. Replace with appropriate Mongo image.

Install chart. Verify deployment afterwards.

```
# helm install fed-kubedb-operator --create-namespace -- namespace
fed-kubedb-operator <chart> -f fed-kubedbvalues.yaml
```

Install fed-service_reg

Provide anyuid and privileged permissions to pods running in `fed-service-reg` project.

```
# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-service-reg

# oc adm policy add-scc-to-group privileged
system:serviceaccounts:fed-service-reg
```

Edit values in `fed-service-values.yaml` as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterwards.

```
# helm install fed-service-reg --create-namespace -- namespace
fed-service-reg <chart> -f fed-servicevalues.yaml
```

Install fed-db_etcd

Provide privileged permissions to pods running in `fed-db-etcd` project.

```
# oc adm policy add-scc-to-group privileged
system:serviceaccounts:fed-db-etcd
```

Edit values in `fed-db-etcd-values.yaml` as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterwards.

```
# helm install fed-db-etcd --create-namespace -- namespace fed-db-etcd <chart> -f fed-dbetcdvalues.yaml
```

Install fed-db_mongo_ram

Edit values in fed-dbmongo-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterwards.

```
# helm install fed-db-mongo-ram --create-namespace --namespace fed-db-mongo-ram <chart> -f fed-dbmongovalues.yaml
```

Install or Enable Observability Components

(Optional) Install OCP cluster logging

Please refer to the official OpenShift documentation on [Installing cluster logging using the CLI](#).

(Optional) Install OCP Jaeger

Please refer to the official OpenShift documentation on [Installing Jaeger](#).

Create a Jaeger instance. Please note the agent strategy is set as "DaemonSet" in the resource file jaeger-instance.yaml.

On OCP when using Daemonset strategy, Jaeger agents won't be allowed to use host ports.

To make it work, create a SCC as shown below:

```
kind: SecurityContextConstraints
apiVersion: security.openshift.io/v1
metadata:
  name: daemonset-with-hostport
  annotations:
    kubernetes.io/description: 'Allows DaemonSets to bind to a well-known
host port'
runAsUser:
  type: RunAsAny
seLinuxContext:
  type: RunAsAny
allowHostPorts: true
```

and grant it to the default serviceaccount in the namespace where jaeger instance will be deployed. In example command below, jaeger-system is the project/namespace where the jaeger-instance is created:

```
# oc adm policy add-scc-to-user daemonset-with-hostport -z default -n jaeger-system
```

(Optional) Enable OCP Monitoring for user workload

By default, monitoring is not enabled for user workload in OpenShift. Enable User Workload Monitoring by creating and modifying the cluster-monitoring-config config map in the openshift-monitoring project/namespace.

Set `enableUserWorkload: true` under `data/config.yaml`.

Edit cluster-monitoring-config

```
# oc -n openshift-monitoring edit configmap cluster-monitoring-config
```

With these changes you should be able to see user workload metrics in Thanos query UI running on openshift-monitoring project.

For more details please refer to [monitoring-for-user-defined-projects](#) and for configuring the resource please refer to [Configuring monitoring resources](#).

***Note:** If you set the Datasource as OCP monitoring Prometheus/Thanos in Grafana then you need to use Thanos Query URL to fetch user workload metrics.*

(Optional) Enable OCP Alerting - Alerting for user defined projects/workloads

If we enable user defined projects monitoring as explained in previous section, it also brings up Thanos ruler for user defined alerts. Thanos ruler evaluates Prometheus alerting rules defined in user projects and sends them to centralized openshift-monitoring Alertmanager. Alerts from user defined projects can be seen in the centralized Alertmanager.

Centralized openshift-monitoring Alertmanager has global configuration to configure alert receivers like webhook, Slack, email and PagerDuty. Once the receiver is configured then you can see alerts in respective receiver.

For more details on receiver configuration please refer to: [sending-notifications-to-external-systems managing-alerts](#).

Install fed-grafana

Provide privileged and anyuid permissions to istio pods running in fed-grafana project.

```
# oc adm policy add-scc-to-group privileged
system:serviceaccounts:fed-grafana
```

```
# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-grafana
```

Edit values in fed-grafana-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterwards.

```
# helm install fed-grafana --create-namespace --namespace fed-
grafana <chart> -f fed-grafana-values.yaml
```

Install Network Function Federations

Install fed-smf

Provide anyuid permissions to pods running in fed-smf project.

```
# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-smf
```

Edit values in fed-smf-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterwards.

```
# helm install fed-smf --create-namespace --namespace fed-smf
<chart> -f fed-smf-values.yaml
```

Install fed-amf

Provide anyuid and privileged permissions to pods running in fed-amf project.

```
# oc adm policy add-scc-to-group system:serviceaccounts:fed-amf

# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-amf
```

Edit values in fed-amf-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterwards.

```
# helm install fed-amf --create-namespace --namespace fed-amf
<chart> -f fed-amf-values.yaml
```

Install fed-nrf

Provide anyuid and privileged permissions to pods running in fed-nrf project.

```
# oc adm policy add-scc-to-group privileged
system:serviceaccounts:fed-nrf

# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-nrf
```

Edit values in fed-nrf-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterwards.

```
# helm install fed-nrf --create-namespace --namespace fed-nrf
<chart> -f fed-nrf-values.yaml
```

Install fed-nssf

Provide anyuid and privileged permissions to pods running in fed-nssf project.

```
# oc adm policy add-scc-to-group privileged
system:serviceaccounts:fed-nssf
```

```
# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-nssf
```

Edit values in fed-nssf-values.yaml as shown in [Helm Values for UnityCloud Federations](#).

Install chart. Verify deployment afterwards.

```
# helm install fed-nssf --create-namespace --namespace fed-nssf
<chart> -f fed-nssf-values.yaml
```

Install fed-upf

Provide anyuid, privileged and hostmount-anyuid permissions to pods running in fed-upf project.

```
# oc adm policy add-scc-to-group privileged
system:serviceaccounts:fed-upf
```

```
# oc adm policy add-scc-to-group anyuid
system:serviceaccounts:fed-upf
```

```
# oc adm policy add-scc-to-group hostmount-anyuid
system:serviceaccounts:fed-upf
```

Edit values in fed-upf-values.yaml as shown in [Helm Values for UnityCloud Federations](#). Make sure to configure other required parameters. Also, make sure to update `prometheusinfo.url` to cluster's OCP Thanos querier route.

Install chart. Verify deployment afterwards.

```
# helm install fed-nssf --create-namespace --namespace fed-upf
<chart> -f fed-upf-values.yaml
```

Configuring deployed network functions

After deploying the network function federations, proceed to configure each network function. Refer to the UnityCloud User Guide for details on configuring each network function after deployment.

Helm Values For UnityCloud Federations

The following are snippets of values.yaml files that are especially relevant to OCP deployments. Make sure to configure all required values before deploying a federation. For a full list of required parameters for each federation, please refer to the UnityCloud Deployment Guide section, "Installing a Federation".

Common Federations

fed-crds

```
pod-prometheus-crd:
  prometheus:
    crd: false
pod-cdpin-crd:
  cni:
    crd: false
    enabled: false
```

fed-metallb

```
global:
  defaultRbacEnabled: true
  defaultClusterRole: permissive-full-cr
  defaultServiceAccountName: default-serviceaccount
pod-metallb:
  rbacEnabled: true
```

For development environments without BGP integration, fed-metallb supports Layer2 load balancing. For example, you can override BGP configuration by doing:

```
pod-metallb:
  configOverride:
    address-pools:
      - name: default
        protocol: layer2
        addresses:
          - "10.0.50.0/24"
```

fed-opa

```
global:
  defaultClusterRole: permissive-network-cr
  defaultRbacEnabled: true
  defaultServiceAccountName: fed-opa-serviceaccount
grafana:
  prometheusinfo:
    url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
```

fed-istio

```
global:
  defaultClusterRole: permissive-network-cr
  defaultRbacEnabled: true
  defaultServiceAccountName: unity-cloud-serviceaccount
grafana:
  prometheusinfo:
    url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
```

```
proxy:
  privileged: true
  overrides:
    preDeleteEnabled: false
pod-istio:
  istioinit:
    useMaistraUbi8Image: true
  rbacEnabled: true
```

fed-paas_helpers

```
global:
  defaultRbacEnabled: true
  defaultClusterRole: permissive-network-cr
  defaultServiceAccountName: fed-paas-helpers-serviceaccount
grafana:
  prometheusinfo:
    url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
  overrides:
    postInstallEnabled: false
logging:
  enabled: false
pod-capture_mgr:
  enabled: true
  rbacEnabled: true
  serviceAccountName: capture-mgr-sa
pod-cert_manager:
  enabled: true
  rbacEnabled: true
  serviceAccount:
    create: true
    name: cert-manager-sa
pod-cores_mgr:
  enabled: true
  rbacEnabled: true
  serviceAccountName: cores-mgr-sa
pod-fluentd_operator:
  enabled: false
  fluentd:
    enabled: false
  prometheusEnabled: false
  rbacEnabled: true
  serviceAccountName: fluentd-operator-serviceaccount
pod-fluentd_operator:
  enabled: false
  fluentd:
    enabled: false
  prometheusEnabled: false
  rbacEnabled: true
  serviceAccountName: fluentd-operator-serviceaccount
pod-grafana_operator:
  deployment:
    authToProm:
      oAuth:
        enabled: true
pod-istio_config:
  rbacEnabled: true
  serviceAccountName: istio-config-controller-sa
pod-jaeger_agent:
  agent:
    enabled: false
  rbacEnabled: true
  serviceAccountName: jaeger-agent-sa
pod-kargo:
  clusterRole: permissive-full-cr
  logLevel: info
```

```

rbacEnabled: true
pod-mongo_cfg_op:
  enabled: true
  rbacEnabled: true
  serviceAccountName: mongo-cfg-op

```

Note: Make sure variable `podgrafana_operator.deployment.authToProm.oAuth.enabled` is set to **true**. This allows `fed-grafana` to authenticate with OCP Prometheus.

fed-kubedb_operator

```

global:
  defaultClusterRole: permissive-network-cr
  defaultRbacEnabled: true
  defaultServiceAccountName: fed-kubedb-operator-serviceaccount
  overrides:
    postUpgradeEnabled: false
    preInstallEnabled: false
    preUpgradeEnabled: false
pod-kubedb_operator:
  rbacEnabled: true
  serviceAccount:
    create: true
    name: kubedb-operator-sa
  mongoServerImage: "<mongo_server_image>"
  catalog:
    mongodb: true

```

fed-service-reg

```

global:
  defaultClusterRole: permissive-full-cr
  defaultRbacEnabled: true
  defaultServiceAccountName: fed-service-reg-serviceaccount
  grafana:
    prometheusinfo:
      url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
    enablePodSecurityPolicies: true
    overrides:
      preDeleteEnabled: false
pod-service_reg:
  rbacEnabled: true

```

fed-db_etcd

```

global:
  defaultClusterRole: permissive-full-cr
  defaultRbacEnabled: true
  defaultServiceAccountName: fed-db-etcd-serviceaccount
  grafana:
    prometheusinfo:
      url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
    overrides:
      preDeleteEnabled: false
pod-etcd_operator:
  rbacEnabled: true
  serviceAccount:
    backupOperatorServiceAccount:
      create: false
      name: backupoperator-sa
    etcdOperatorServiceAccount:
      create: true

```

```
    name: etcdoperator-sa
  restoreOperatorServiceAccount:
    create: false
    name: restoreoperator-sa
```

fed-db_mongo_ram

```
global:
  defaultRbacEnabled: true
  defaultClusterRole: permissive-network-cr
  defaultServiceAccountName: db-percona-serviceaccount
  grafana:
    prometheusinfo:
      url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
  overrides:
    postInstallEnabled: false
  pod-db_percona_mongo:
    rbacEnabled: true
```

Observability Federations

fed-prometheus

```
global:
  overrides:
    preDeleteEnabled: false
    postInstallEnabled: false
  pod-prometheus:
    promInstanceSelector: true
```

fed-grafana

```
global:
  defaultRbacEnabled: true
  defaultClusterRole: permissive-network-cr
  defaultServiceAccountName: fed-grafana-serviceaccount
  grafana:
    prometheusinfo:
      url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
  overrides:
    preDeleteEnabled: false
    postInstallEnabled: false
  pod-grafana:
    enabled: true
    rbacEnabled: true
    serviceAccountName: "grafana-serviceaccount"
```

Network Federations

fed-smf

```
global:
  defaultRbacEnabled: true
  defaultClusterRole: permissive-network-cr
  defaultServiceAccountName: fed-smf-serviceaccount
  grafana:
    prometheusinfo:
      url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
  ingressNetworkPolicy:
    enabled: false
```

fed-amf

```
global:
```

```

defaultRbacEnabled: true
defaultClusterRole: permissive-full-cr
defaultServiceAccountName: fed-amf-serviceaccount
grafana:
  prometheusinfo:
    url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
ingressNetworkPolicy:
  enabled: false

```

fed-nrf

```

global:
  defaultRbacEnabled: true
  defaultClusterRole: permissive-network-cr
  defaultServiceAccountName: fed-nrf-serviceaccount
  grafana:
    prometheusinfo:
      url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
  ingressNetworkPolicy:
    enabled: false

```

fed-nssf

```

global:
  defaultRbacEnabled: true
  defaultClusterRole: permissive-network-cr
  defaultServiceAccountName: fed-nssf-serviceaccount
  grafana:
    prometheusinfo:
      url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
  ingressNetworkPolicy:
    enabled: false

```

fed-upf

```

global:
  defaultRbacEnabled: true
  defaultClusterRole: permissive-network-cr
  defaultServiceAccountName: fed-nrf-serviceaccount
  grafana:
    prometheusinfo:
      url: "https://thanos-querier-openshiftmonitoring.apps.test.lab.example.com"
  overrides:
    userClusterAdminRole: true
  ingressNetworkPolicy:
    enabled: false

```